

Operation systems& Adv. Mobile project

Homework 2

Multithreading

Made by: Otcheskii Leonid (32239236)

Notation to the program:

Task 1 was completed using a mutex *lock* and a condition variable *cond*. To prevent data leakage, the *free(ret)* operation at the end of each consumer cycle was added.

For the concurrent producer/consumer operations (**Task 2**) a ring-buffer was used. Semaphores were essential to protect the critical sections. The '*full*' semaphore keeps track of how many slots in the ring-buffer are filled, and the '*empty*' semaphore shows how many slots are available. The size of the ring-buffer is set by a specific variable. One critical point: the maximum number of consumers or producers can't be more than the buffer size, or we'll run into a deadlock.

To gather the statistics (**Task 3**) '*Stat_analys*' function was implemented. This function is using within consumer threads. To store statistics and provide an access to all consumers additional Shared object *stats* was defined. The object includes '*stat1*' to count words by length and '*stat2*' to keep a running total of all the letters. to make sure the data stays correct and protected when it's being accessed concurrently mutex lock was additionally defined.

PS: To reduce the completion time all the outputs within consumers/producers threads were commented.

Program tests

Task 1:

1 consumer & 1 producer;

File: FreeBSD9-orig.tar

1) With Cons./Prod. Output uncommented:

```
real    6m54.018s
user    2m27.380s
sys     3m37.537s
```

2) Without Cons./Prod. Output

```
main continuing
main: consumer_0 joined with 21262918
main: producer_0 joined with 21262918

real    4m19.588s
user    1m53.395s
sys     2m39.080s
```

Conclusion: I/O operations in thread make a sufficient impact in working time.

Task 2:
File: FreeBSD9-orig.tar

1) 1 consumer + 1 producer (without thread output)

real	0m23.941s
user	0m25.317s
sys	0m13.664s

2) 50 consumer + 50 producers (without thread output)

real	0m33.042s
user	1m56.023s
sys	11m14.054s

3) 10 consumers + 10 producers (without thread output)

real	0m28.678s
user	1m25.564s
sys	7m6.208s

4) 100 consumers + 100 producers (without thread output)

real	0m36.366s
user	2m19.613s
sys	12m9.551s

Conclusion: Sometimes amount of threads doesn't guarantee the working speed increasing.

Task 3:
File: FreeBSD9-orig.tar

*** Character statistics ***	*** Word length distribution ***
A: 24461614	Length 1: 12499260
B: 8093992	Length 2: 11541548
C: 18556186	Length 3: 9170747
D: 16466922	Length 4: 10342651
E: 39758123	Length 5: 5000577
F: 13002010	Length 6: 9907805
G: 6734619	Length 7: 4530514
H: 8906383	Length 8: 3891007
I: 25609995	Length 9: 2673334
J: 590184	Length 10: 3213947
K: 2836338	Length 11: 1502238
L: 15330830	Length 12: 1195846
M: 9889498	Length 13: 1011957
N: 22722612	Length 14: 821582
O: 20803479	Length 15: 752459
P: 11540733	Length 16: 604860
Q: 842309	Length 17: 476603
R: 23286746	Length 18: 457823
S: 23844047	Length 19: 353180
T: 30560895	Length 20: 352511
U: 10794017	Length 21: 266901
V: 4540040	Length 22: 216479
W: 3450958	Length 23: 192081
X: 11301267	Length 24: 160833
Y: 4242881	Length 25: 134201
Z: 996555	Length 26: 104878
	Length 27: 91846

	Length 28: 82436
	Length 29: 64247
	Length 30: 582876

1) 5 consumers and 5 producers

real	0m45.708s
user	2m31.399s
sys	2m43.371s